couplr: Optimal pairing and matching via linear assignment

Gilles Colling

2 November 2026

Summary

`couplr` (Colling 2026) is an R package for pairing objects from two sets while minimizing their total dissimilarity. It supports matching treated and control subjects, pairing ecological samples, and assigning tasks to resources. Large matching problems can require more memory for pairwise distances than for the observations themselves. `couplr` reduces this storage requirement by computing distances when needed or retaining only candidate pairs, and checks whether the returned assignment is optimal for the complete problem. Users can construct matches from data frames or supply a cost matrix, impose restrictions on eligible pairs, and assess the balance of the resulting groups.

Statement of need

Matching two samples requires choosing among many possible pairs. With n observations in one sample and m in the other, a full distance matrix contains nm entries. Storing these distances can become the main computational constraint as sample sizes increase. Restricting the search to nearby candidates reduces storage, but an assignment that is optimal among those candidates may still be improved by pairs omitted from the search.

`couplr` addresses this problem by combining reduced cost storage with explicit solution verification. Researchers can solve assignments without retaining the full distance matrix and check the result against the complete set of admissible pairs. Calipers and blocking specify which pairs are admissible; verification establishes the quality of the assignment under those restrictions. Balance diagnostics then assess whether the resulting groups provide the intended statistical comparison.

State of the field

Existing R packages support both statistical matching and direct assignment. `MatchIt` (Ho et al. 2011) provides matching and subclassification, including optimal pair and full matching through `optmatch` and genetic matching through `Matching`. `optmatch` (Hansen 2007) supports network-flow matching and sparse distance structures; `Matching` (Sekhon 2011) provides multivariate and propensity-score matching; `designmatch` (Zubizarreta et al. 2024) supports explicit balance constraints. `cobalt` (Greifer 2025) provides diagnostics across packages. For supplied cost matrices, `clue::solve_LSAP()` solves linear assignment and `lpSolve` supports linear and integer programming (Hornik 2024; Berkelaar et al. 2024).

Reducing storage while checking omitted pairs requires distance evaluation to remain available during the solve. `couplr` therefore couples its distance representation to an assignment layer that can generate candidates, identify omitted pairs that could improve a solution, and verify the result against the complete problem. Integrating candidate generation with the assignment solver gives the search direct access to distance calculations and dual values as the candidate graph expands. Conversion methods and registered `cobalt` methods connect its results to existing statistical workflows.

Software design

Figure 1 outlines the matching workflow, from data and restrictions to matched pairs and diagnostics.

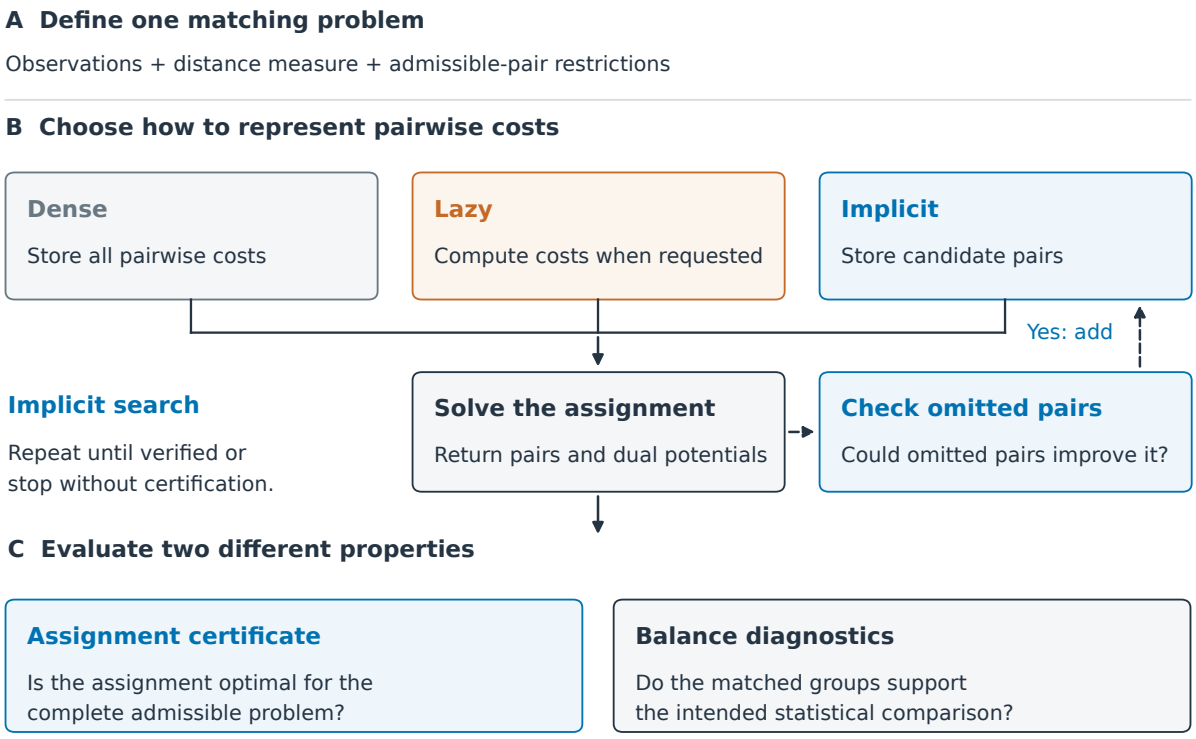


Figure 1: Matching with reduced storage and explicit verification. Three cost representations serve the same objective. In implicit mode, omitted-pair checks expand the candidate set until verification succeeds or the search stops without certification. Assignment certificates and balance diagnostics assess different properties.

The design separates the matching objective from the way costs are stored. Users first specify variables, a distance measure, and restrictions on eligible pairs. Calipers limit differences in individual variables, maximum distances limit overall dissimilarity, and blocking partitions the observations into separate assignments. Available distances include Euclidean, Manhattan, and Mahalanobis distances, alongside user-supplied functions.

For one-to-one assignment, orient the problem so that $n \leq m$, and let E contain the admissible pairs with finite costs C_{ij}. The solver seeks a binary matrix X satisfying

$$\min_X \sum_{(i,j)\in E} C_{ij}X_{ij}, \qquad \sum_{j:(i,j)\in E} X_{ij} = 1 \ \forall i, \qquad \sum_{i:(i,j)\in E} X_{ij} \leq 1 \ \forall j.$$

Every row must receive a distinct partner. When the restrictions make this impossible, a maximum-cardinality matching instead identifies how many rows can receive partners; it is not a feasible solution of the complete-assignment problem.

Three storage modes offer different memory and computation trade-offs. Dense mode stores the full cost matrix. Lazy mode computes distances from the features when requested, saving storage at the expense of repeated distance evaluation. Implicit mode initially retains a restricted set of candidate pairs, solves that assignment, and checks whether omitted pairs could improve it. Improving candidates are added until the complete problem is verified or the search terminates without certification. The certificate records which conclusion was reached. Checking omitted pairs can require repeated scans of the complete pair set, so storage savings need not translate into fewer distance evaluations.

Verification checks whether any admissible assignment could have a lower total cost. Dual potentials supplied by `assignment_duals()` define a lower bound, and `verify_assignment()` checks feasibility and whether the assignment attains that bound. Floating-point expansions support exact checks (Shewchuk 1997). The certificate distinguishes exact optimality for the supplied costs from a numerical conclusion within a stated tolerance. Passing the potentials with the assignment avoids a second solve to obtain them; the benchmark documentation gives the dual conditions and exact-potential recovery procedure.

Separating solver selection from storage allows users to change memory requirements while retaining the matching objective. Jonker–Volgenant (Jonker and Volgenant 1987) is the general default: regime benchmarks did not support a consistent benefit from selecting solvers by sparsity or matrix shape alone. Automatic selection also uses specialized routines for very small matrices and constant or binary costs. Alternative solvers, k-best (Murty 1968) and bottleneck assignments, and additional statistical matching designs are documented with the package.

The returned pairs can then be assessed as a statistical design. Diagnostics report standardized mean differences, variance ratios, and Kolmogorov–Smirnov statistics; Rosenbaum bounds assess sensitivity to unmeasured confounding for supported paired analyses (Rosenbaum 2002). These checks complement assignment verification because a minimum-distance matching need not balance every covariate. Outcome estimation remains a separate step that must reflect the chosen design (Stuart 2010).

Research impact statement

The reproducible benchmarks demonstrate substantial memory savings while retaining checkable solutions. In a separate memory experiment with 20,000 observations, implicit mode reduced peak resident memory above the loaded-session baseline from 1,534.5 MB in dense mode to 137.1 MB, while elapsed time increased from 3.79 to 4.38 seconds. Lazy mode reduced memory further, to 25.5 MB, but required 6.46 seconds. These individual runs illustrate the choice between storing and recomputing distances (Figure 2). In a separate experiment, implicit assignments were certified at all six sizes from 500 to 50,000 observations and agreed in objective value with lazy mode. At the largest size, implicit mode stored 1.60 million of 555.56 million possible pairs.

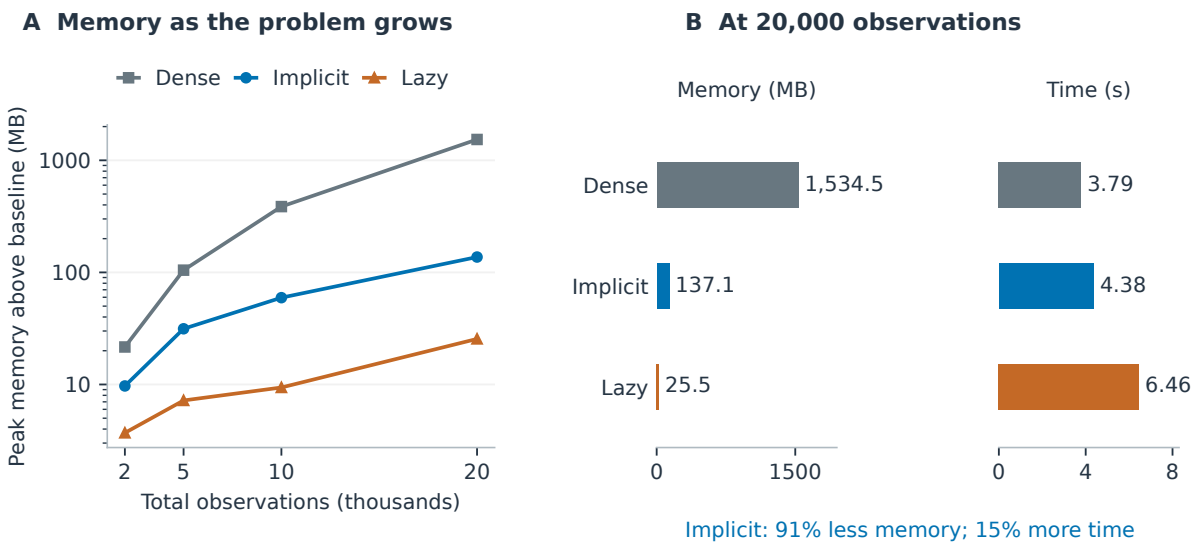


Figure 2: Memory savings and their runtime cost. (A) Peak resident memory above the loaded-session baseline, on a logarithmic scale. (B) Memory and elapsed time at 20,000 observations, on linear scales. Each measurement is an individual run with approximately one treated unit per two controls; percentages compare implicit with dense mode.

The storage advantage depends on the geometry of the data. A broader experiment varies geometry, dimension, sample size, and calipers. The following results are medians across five instances with 10,000 observations and eight covariates. The heavy-tailed case retained 1.17% of possible pairs but took about as long as lazy mode. In the contested case, where observations compete for nearby partners, the candidate graph expanded to 73.3% of possible pairs and distance evaluations reached 10.3 times the number of possible pairs. Implicit mode nevertheless completed in 139 seconds compared with 423 seconds for lazy mode. Storage savings, distance evaluations, and runtime therefore describe distinct aspects of performance.

Exact certificates confirmed optimality on all 120 common-cost validation matrices, covering integer, uniform, and Euclidean costs in square and rectangular problems. Recorded objective values agreed with `clue` and SciPy on all instances and with `lpSolve` on the 60 smaller square instances tested. These comparisons complement small-instance brute-force tests. The accompanying synthetic hospital-staff example demonstrates how verification handles restrictions: implicit mode matches all 200 treated units using 10,800 of 60,000 possible pairs, while tighter restrictions leave at most 197 units matchable. An infeasibility witness identifies a set of rows with too few admissible partners to cover them all.

Certification adds modest runtime to the dense matching workflow. For 20,000 observations, dense `couplr` completed one-to-one Mahalanobis matching in 3.78 seconds, or 4.61 seconds with certification, compared with 288.50 seconds for `optmatch` (Table 1). These timings include distance construction and output preparation; `MatchIt` delegates the optimization to `optmatch`. The benchmark uses pooled within-group covariance and materializes distances in all workflows.

Table 1: Complete matching workflow times in seconds, including distance construction. Medians across independent instances, using the fastest repetition per instance. The second timing column adds solution verification.

Total observations	`couplr`	With certificate	`optmatch`	`MatchIt`
500	0.0033	0.0037	0.0965	0.1161
2,000	0.0331	0.0405	1.6336	2.0696
5,000	0.2089	0.2528	12.0799	14.9241
10,000	0.9299	1.1649	58.7787	71.3423
20,000	3.7817	4.6087	288.4989	overflow
50,000	25.0925	31.2177	timeout	timeout

Treated:control is approximately 1:2, with eight covariates and three to five independent instances per size. All workflows materialize distances. Runs use one core and single-threaded BLAS on an Apple M4 Pro, R 4.5.3, `couplr` 1.9.0, `optmatch` 0.10.8, and `MatchIt` 4.7.2. The optmatch size limit is disabled. `overflow` denotes a size-limit error reached through MatchIt; `timeout` denotes a feasibility probe exceeding 600 seconds. The benchmark documentation provides the repetition counts, timing protocol, error text, and quartiles across instances.

An optimal assignment can still reveal a limitation of the statistical design. In the LaLonde job-training example (LaLonde 1986; Dehejia and Wahba 1999), matching all 185 treated units to distinct controls from 429 available candidates leaves an absolute standardized mean difference of 1.053 for the indicator for Black participants. The treated sample includes 156 Black participants, but only 87 Black controls are available. This shortage makes balance on the indicator unattainable under complete one-to-one matching, regardless of the solver. Balance agrees to three decimal places across `couplr`, `MatchIt`, and `optmatch`, despite a small difference in assignment cost. The diagnostic shows how much imbalance remains in the selected pairs (Figure 3).

Scripts, individual measurements, environment records, and extended examples accompany the paper under `paper/bench`. The scaling table is drawn from `results/scaling-results.csv`; the benchmark documentation provides the timing protocol, variation across instances, and solver settings.

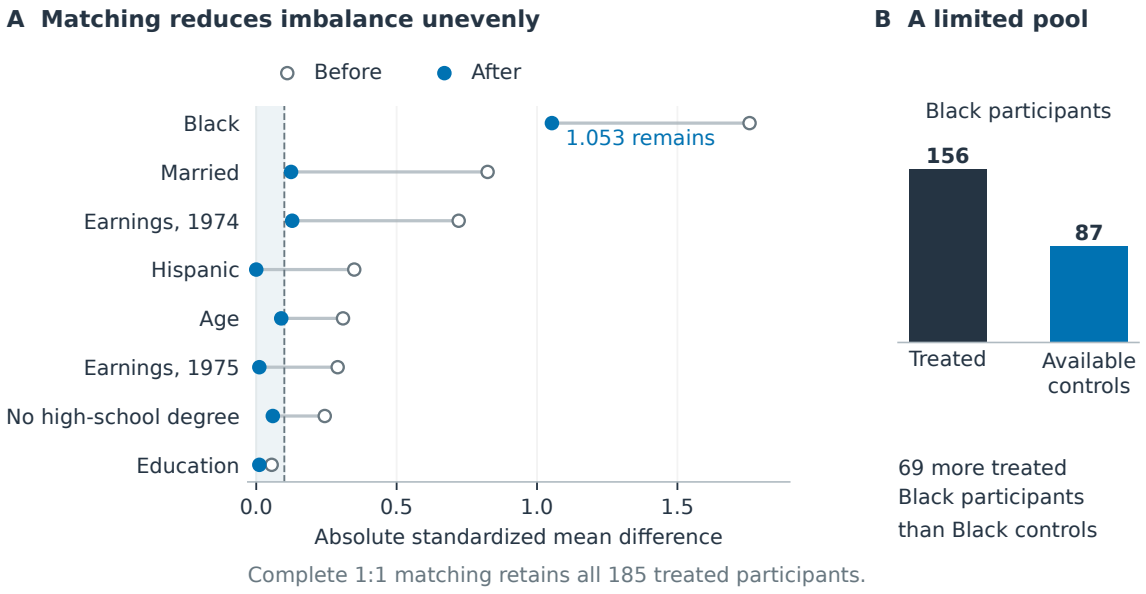


Figure 3: Optimal matching does not ensure covariate balance. (A) LaLonde covariate balance before and after complete one-to-one matching; the shaded band and dashed line mark absolute standardized mean differences up to 0.1. Matched values agree to three decimal places across the three packages. (B) The shortage of Black controls prevents balance on this indicator when all treated participants must be retained.

AI usage disclosure

The author developed the software with AI assistance, designed its architecture, and selected its algorithms. A self-hosted Qwen3-Coder-Next-REAP-48B-A3B model, accessed through the Claude Code CLI, assisted the author's development through code completion, refactoring, and boilerplate generation. Large language models accessed through u:ai, the University of Vienna's AI service, supported grammar and structure checking and manuscript revision. The author reviewed and edited the suggested wording and code and takes full responsibility for the code, results, and conclusions. Validation included brute-force comparisons, external-solver comparisons, and explicit optimality checks.

Acknowledgements

No external funding supported this work.

References

Berkelaar, Michel et al. 2024. *lpSolve: Interface to Lp_solve v. 5.5 to Solve Linear/Integer Programs*. https://CRAN.R-project.org/package=lpSolve.

Colling, Gilles. 2026. *Couplr: Optimal Pairing and Matching via Linear Assignment*. https://doi.org/10.32614/CRAN.package.couplr.

Dehejia, Rajeev H., and Sadek Wahba. 1999. "Causal Effects in Nonexperimental Studies: Reevaluating the Evaluation of Training Programs." *Journal of the American Statistical Association* 94 (448): 1053–62. https://doi.org/10.1080/01621459.1999.10473858.

Greifer, Noah. 2025. "cobalt: Covariate Balance Tables and Plots." *Journal of Open Source Software* 10 (109): 7610. https://doi.org/10.21105/joss.07610.

Hansen, Ben B. 2007. "Optmatch: Flexible, Optimal Matching for Observational Studies." *R News* 7 (2): 18–24. https://journal.r-project.org/articles/RN-2007-014/.

Ho, Daniel E., Kosuke Imai, Gary King, and Elizabeth A. Stuart. 2011. "MatchIt: Nonparametric Preprocessing for Parametric Causal Inference." *Journal of Statistical Software* 42 (8): 1–28. https://doi.org/10.18637/jss.v042.i08.

Hornik, Kurt. 2024. *Clue: Cluster Ensembles*. https://CRAN.R-project.org/package=clue.

Jonker, Roy, and Ton Volgenant. 1987. "A Shortest Augmenting Path Algorithm for Dense and Sparse Linear Assignment Problems." *Computing* 38 (4): 325–40. https://doi.org/10.1007/BF02278710.

LaLonde, Robert J. 1986. "Evaluating the Econometric Evaluations of Training Programs with Experimental Data." *The American Economic Review* 76 (4): 604–20. https://www.jstor.org/stable/1806062.

Murty, Katta G. 1968. "An Algorithm for Ranking All the Assignments in Order of Increasing Cost." *Operations Research* 16 (3): 682–87. https://doi.org/10.1287/opre.16.3.682.

Rosenbaum, Paul R. 2002. *Observational Studies*. 2nd ed. Springer Series in Statistics. Springer. https://doi.org/10.1007/978-1-4757-3692-2.

Sekhon, Jasjeet S. 2011. "Multivariate and Propensity Score Matching Software with Automated Balance Optimization: The Matching Package for R." *Journal of Statistical Software* 42 (7): 1–52. https://doi.org/10.18637/jss.v042.i07.

Shewchuk, Jonathan Richard. 1997. "Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates." *Discrete & Computational Geometry* 18 (3): 305–63. https://people.eecs.berkeley.edu/~jrs/papers/robustr.pdf.

Stuart, Elizabeth A. 2010. "Matching Methods for Causal Inference: A Review and a Look Forward." *Statistical Science* 25 (1): 1–21. https://doi.org/10.1214/09-STS313.

Zubizarreta, José R., Cinar Kilcioglu, and Juan P. Vielma. 2024. *Designmatch: Matched Samples That Are Balanced and Representative by Design*. https://CRAN.R-project.org/package=designmatch.